

HW4
Statistical Method
H24101222_陳凱騫
2024-11-20

Table of contents

Load datasets Set1.csv and Set2.csv	2
EDA	3
(1.a) Use the method of moments (MME) to estimate the model parameters.	4
Normal	4
Set1	4
Set2	5
Exponential distribution	6
Set1	6
Set2	7
Laplace distribution	8
Set1	9
Set2	10
Gamma distribution	11
Set1	12
Set2	13
Conclusion	14
(1.b) Use the maximum likelihood estimates (MLEs) to estimate the model parameters.	15
Normal	15
Set1	15
Set2	16
Exponential	17
Set1	17
Set2	18
Laplace	18
Set1	19
Set2	20
Gamma	21
Set1	21
Set2	22
Conclusion	23
(1.c) Are the MMEs and MLEs the same for each distribution?	23
(1.d) and (1.e)	23
(1.f)	24

dataset1	24
normal	24
laplace	24
dataset2	25
normal	25
gamma	25
(1.g)	25
dataset1	25
dataset2	27
iris data for question 2	28
Load datasets iris.csv	28
EDA	28
(2.a)	28
(2.b)	30
(2.c)	30
(2.d)	30
(2.e)	31

Load datasets Set1.csv and Set2.csv

```
library(knitr)
data1 <- read.csv("Set1.csv")
head(data1)
```

```
      x
1 -0.04517973
2  0.09302886
3  0.13015753
4  0.02498463
5 -0.81960729
6  2.51160132
```

```
data2 <- read.csv("Set2.csv")
head(data2)
```

```
      x
1 21.34629
2 31.87284
3 17.92697
4 21.16752
5 18.96388
6 18.29954
```

EDA

```

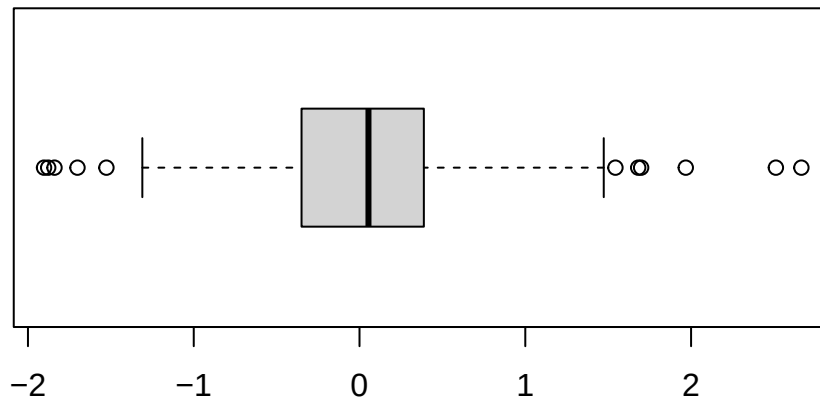
data1
1 Variables      100 Observations
x
n missing distinct  Info  Mean  Gmd  .05  .10  .25  .50
100      0      100    1 0.02471  0.9118 -1.3208 -1.0350 -0.3470  0.0540
.75      .90      .95
0.3841  0.8573  1.5506
lowest: -1.90298 -1.8779 -1.83993 -1.70134 -1.52673, highest: 1.68114 1.69871 1.96807 2.5116 2.66534

```

data2												
1 Variables 30 Observations												
x												
n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95
30	0	30	1	19.25	5.926	11.99	13.32	15.73	18.63	22.28	25.35	27.15
lowest : 8.62877 11.8175 12.1968 13.4407 13.9686, highest: 24.98 25.2753 26.057 28.0409 31.8728												

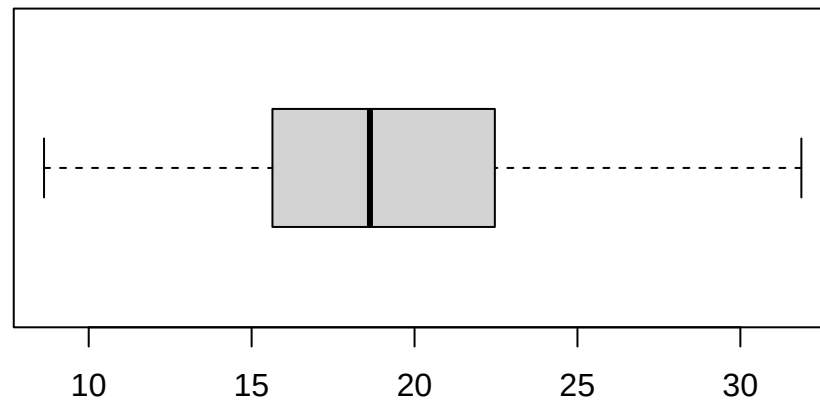
```
boxplot(c(data1),horizontal=TRUE,main="boxplot of Set1")
```

boxplot of Set1



```
boxplot(c(data2),horizontal=TRUE,main="boxplot of Set2")
```

boxplot of Set2



(1.a) Use the method of moments (MME) to estimate the model parameters.

Normal

- The decision variables for the normal distribution are μ and σ^2 .
- Objective function: $E(X) = \mu$ and $E(X^2) = \sigma^2 + \mu^2$
- Theoretical derivation / efficient algorithms:

$$\hat{\mu}_{MME} = \frac{1}{n} \sum_{i=1}^n x_i$$

and

$$\hat{\sigma}_{MME}^2 = \frac{1}{n} \sum_{i=1}^n x_i^2 - \hat{\mu}_{MME}^2$$

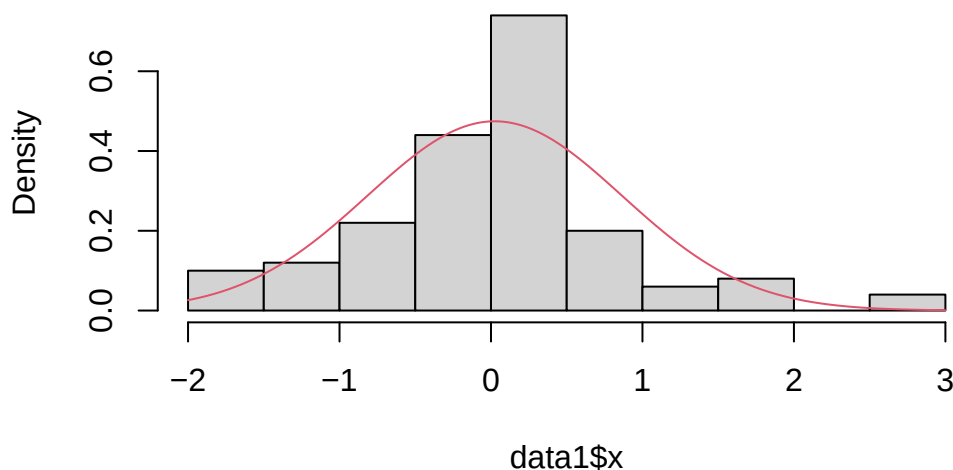
Set1

```
n <- nrow(data1) # 樣本數量
EX <- mean(data1$x) # 一次動差
EX2 <- mean(data1$x^2) # 二次動差
mu.mme <- EX
sigma2.mme <- EX2 - (EX)^2
print(c(mu.mme, sigma2.mme))
```

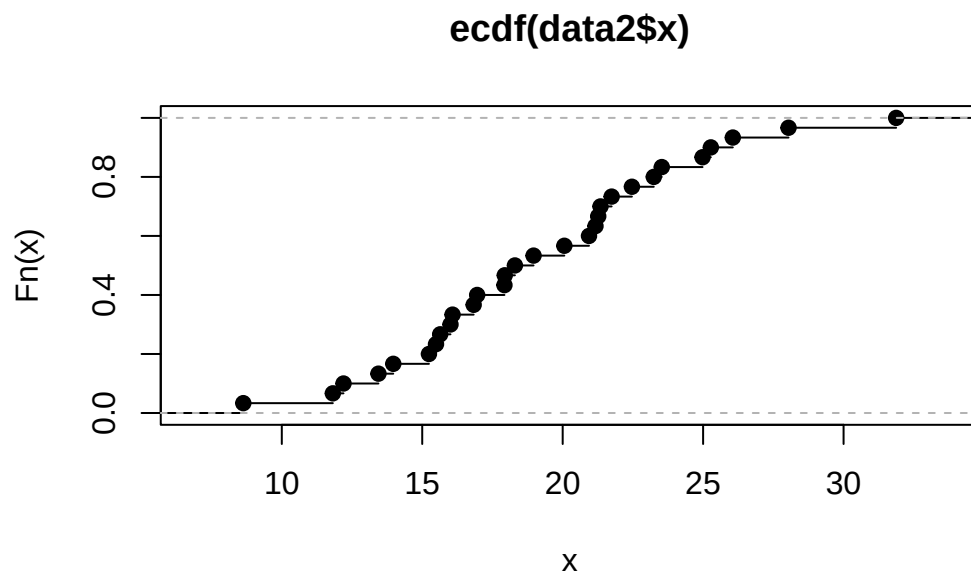
```
[1] 0.02471087 0.70672248
```

```
### Verification: By fitting plot
hist(data1$x, probability = TRUE)
x0 <- seq(-2, 3, 0.01)
lines(x0, dnorm(x0, mu.mme, sqrt(sigma2.mme)), col = 2)
```

Histogram of data1\$x



```
plot(ecdf(data2$x))
lines(x0, pnorm(x0, mu.mme, sqrt(sigma2.mme)), col = 2)
```

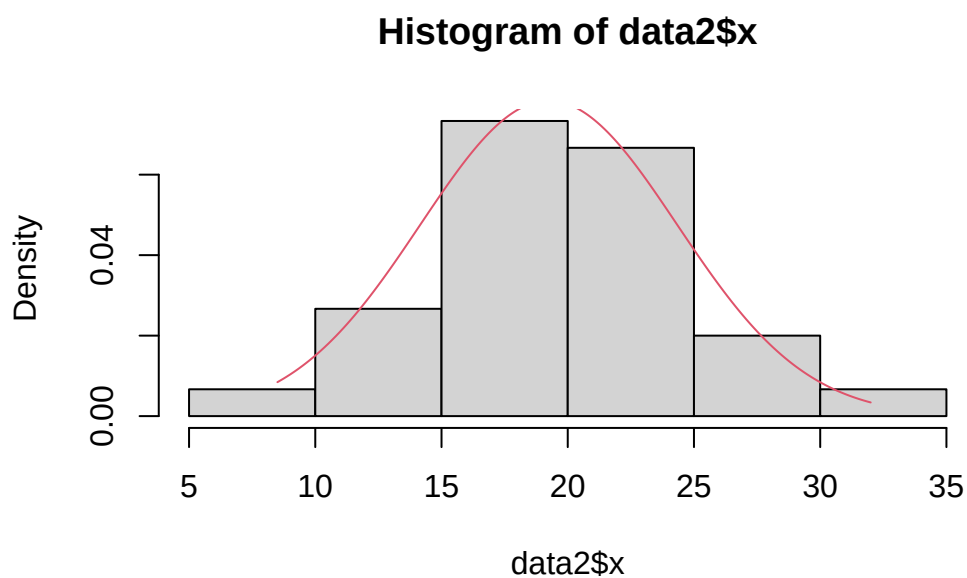


Set2

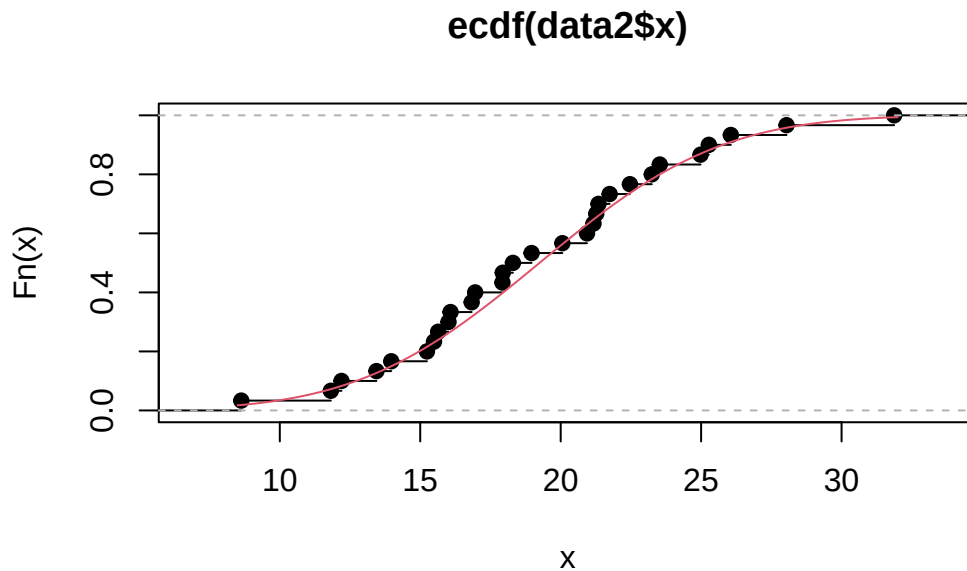
```
n <- nrow(data2) # 樣本數量
EX <- mean(data2$x) # 一次動差
EX2 <- mean(data2$x^2) # 二次動差
mu.mme <- EX
sigma2.mme <- EX2 - (EX)^2
print(c(mu.mme, sigma2.mme))
```

```
[1] 19.24671 25.91264
```

```
### Verification: By fitting plot
hist(data2$x, probability = TRUE)
x0 <- seq(8.5, 32, 0.01)
lines(x0, dnorm(x0, mu.mme, sqrt(sigma2.mme)), col = 2)
```



```
plot(ecdf(data2$x))
lines(x0, pnorm(x0, mu.mme, sqrt(sigma2.mme)), col = 2)
```



Exponential distribution

- The decision variable for the exponential distribution is λ .
- Objective function: $E(X) = \frac{1}{\lambda}$
- Theoretical derivation / efficient algorithms:

$$\hat{\lambda}_{MME} = \frac{n}{\sum_{i=1}^n x_i}$$

Set1

```
n <- nrow(data1) # 樣本數量
EX <- mean(data1$x) # 一次動差
lambda.mme <- 1/EX
cat("lambda.mme是:")
```

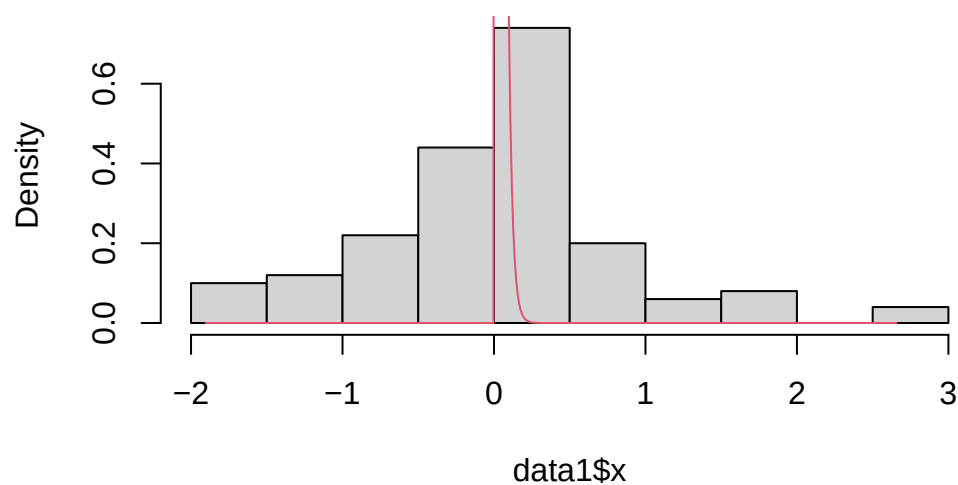
lambda.mme是:

```
print(lambda.mme)
```

```
[1] 40.46803
```

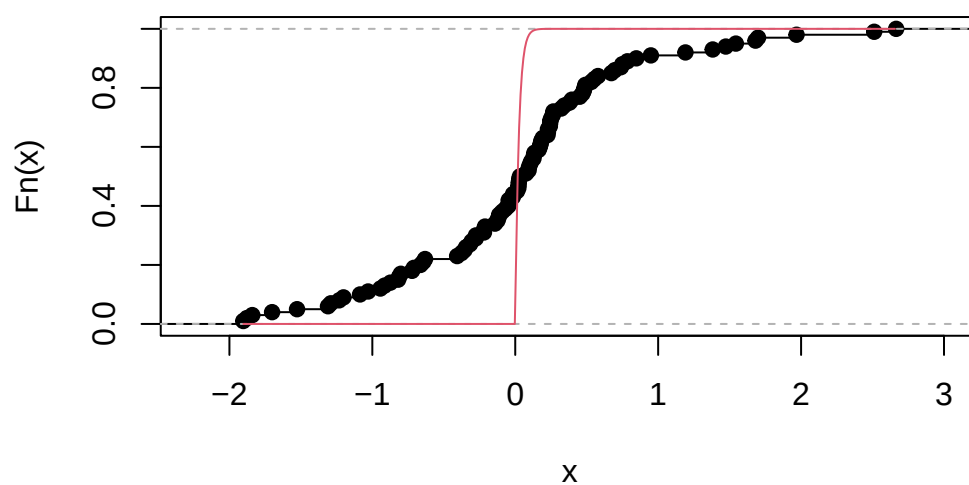
```
### Verification: By fitting plot
hist(data1$x, probability = TRUE)
x0 <- seq(min(data1$x), max(data1$x), 0.01)
lines(x0, dexp(x = x0, rate = lambda.mme), col = 2)
```

Histogram of data1\$x



```
plot(ecdf(data1$x))
lines(x0, pexp(x0, rate = lambda.mme), col = 2)
```

ecdf(data1\$x)



Set2

```
n <- nrow(data2) # 樣本數量
EX <- mean(data2$x) # 一次動差
lambda.mme <- 1/EX
cat("lambda.mme是:")
```

lambda.mme是:

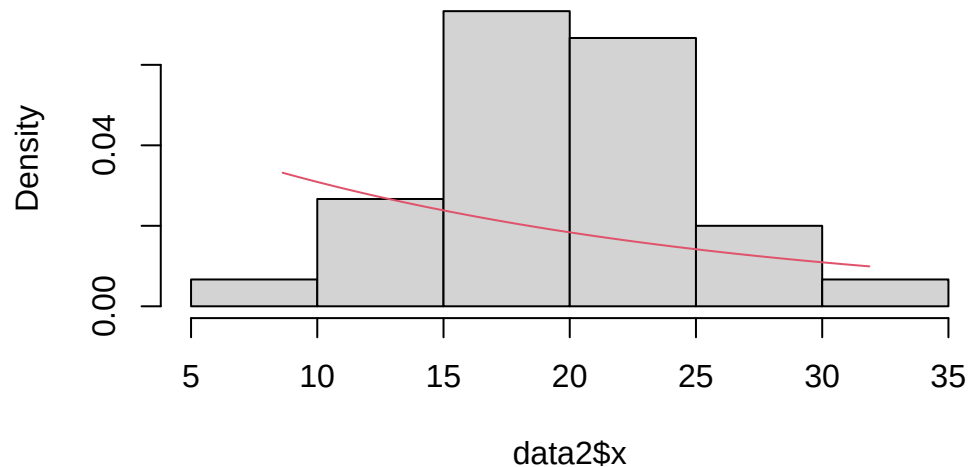
```
print(lambda.mme)
```

```
[1] 0.05195693
```

```
### Verification: By fitting plot
hist(data2$x, probability = TRUE)
```

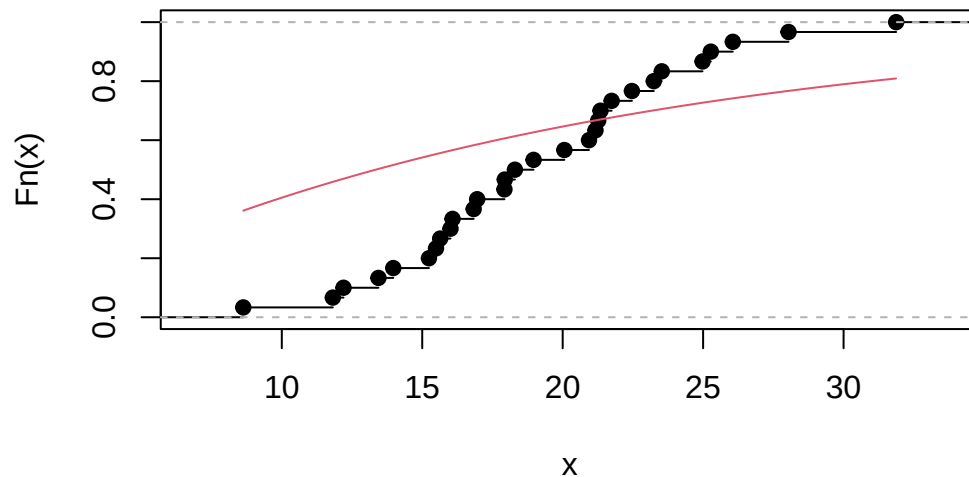
```
x0 <- seq(min(data2$x), max(data2$x), 0.01)
lines(x0, dexp(x = x0, rate = lambda.mme), col = 2)
```

Histogram of data2\$x



```
plot(ecdf(data2$x))
lines(x0, pexp(x0, rate = lambda.mme), col = 2)
```

ecdf(data2\$x)



Laplace distribution

- The decision variables for the normal distribution are μ and b .
- Objective function: $E(X) = \mu$ and $E(X^2) = \sigma^2 + \mu^2$
- Theoretical derivation / efficient algorithms:

$$\hat{\mu}_{MME} = \frac{\sum_{i=1}^n x_i}{n}$$

$$\hat{b}_{MME} = \sqrt{\frac{1}{2} \left(\frac{1}{n} \sum_{i=1}^n X_i^2 - \left(\frac{1}{n} \sum_{i=1}^n X_i \right)^2 \right)}$$

Set1

```
n <- nrow(data1) # 樣本數量
EX <- mean(data1$x) # 一次動差
EX2 <- mean(data1$x^2) # 二次動差
mu.mme <- EX
b.mme <- sqrt(0.5*(EX2-EX^2))
cat("mu.mme是:")
```

mu.mme是:

```
print(mu.mme)
```

```
[1] 0.02471087
```

```
cat("b.mme是:")
```

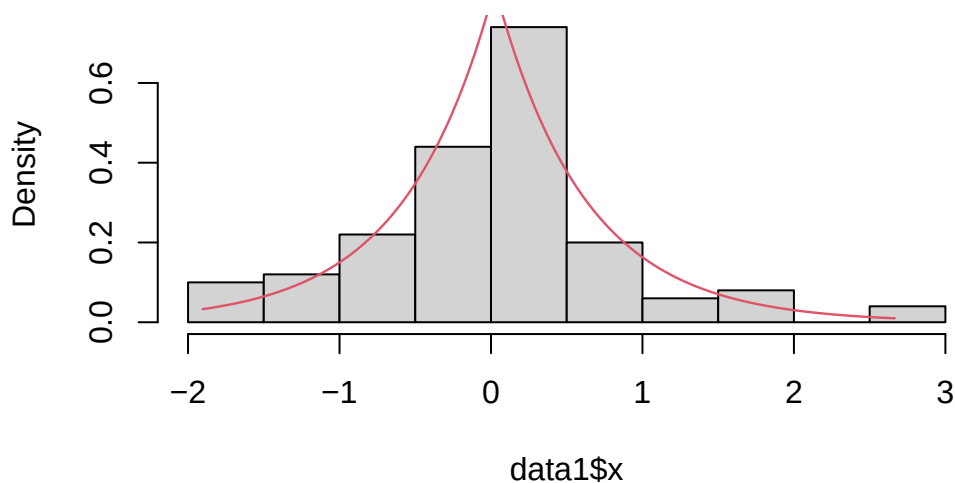
b.mme是:

```
print(b.mme)
```

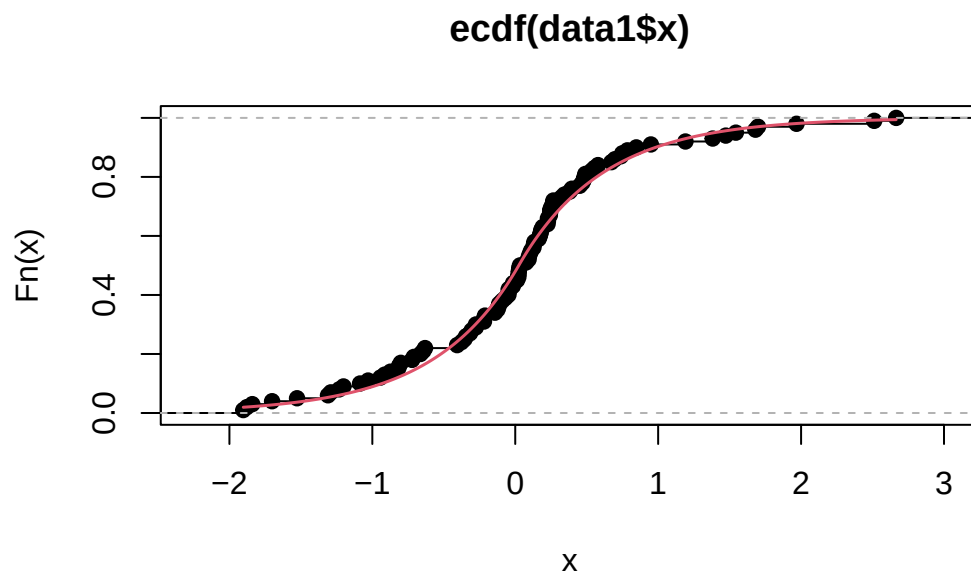
```
[1] 0.594442
```

```
### Verification: By fitting plot
# install.packages("extraDistr")
library(extraDistr)
hist(data1$x, probability = TRUE)
x0 <- seq(min(data1$x), max(data1$x), 0.001)
lines(x0, dlaplace(x0, mu = mu.mme, sigma = b.mme), col = 2, lwd = 1.2)
```

Histogram of data1\$x



```
plot(ecdf(data1$x))
lines(x0, plaplace(x0, mu = mu.mme, sigma = b.mme), col = 2, lwd = 1.5)
```



Set2

```
n <- nrow(data2) # 樣本數量
EX <- mean(data2$x) # 一次動差
EX2 <- mean(data2$x^2) # 二次動差
mu.mme <- EX
b.mme <- sqrt(0.5*(EX2-EX^2))
cat("mu.mme是:")
```

mu.mme是:

```
print(mu.mme)
```

```
[1] 19.24671
```

```
cat("b.mme是:")
```

b.mme是:

```
print(b.mme)
```

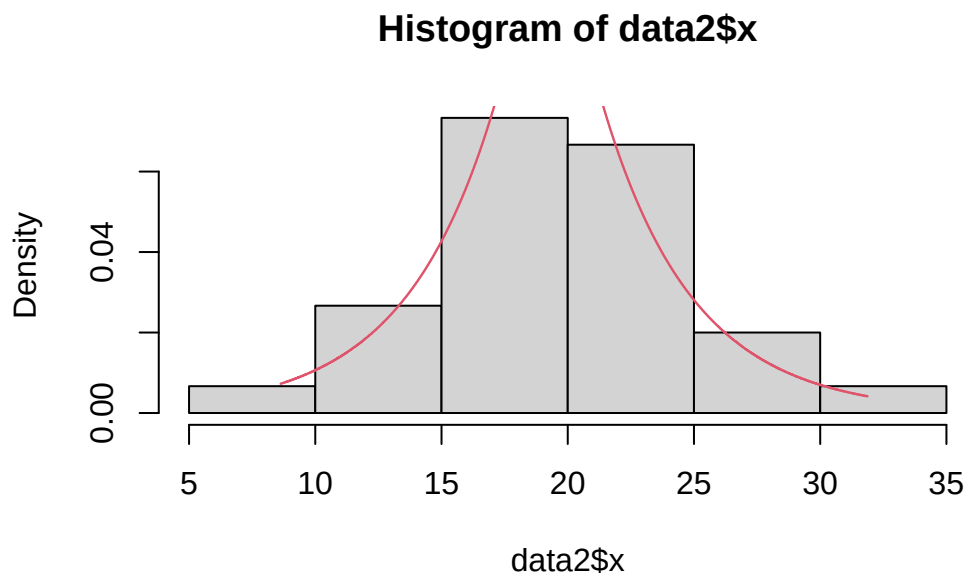
```
[1] 3.599489
```

```
### Verification: By fitting plot
```

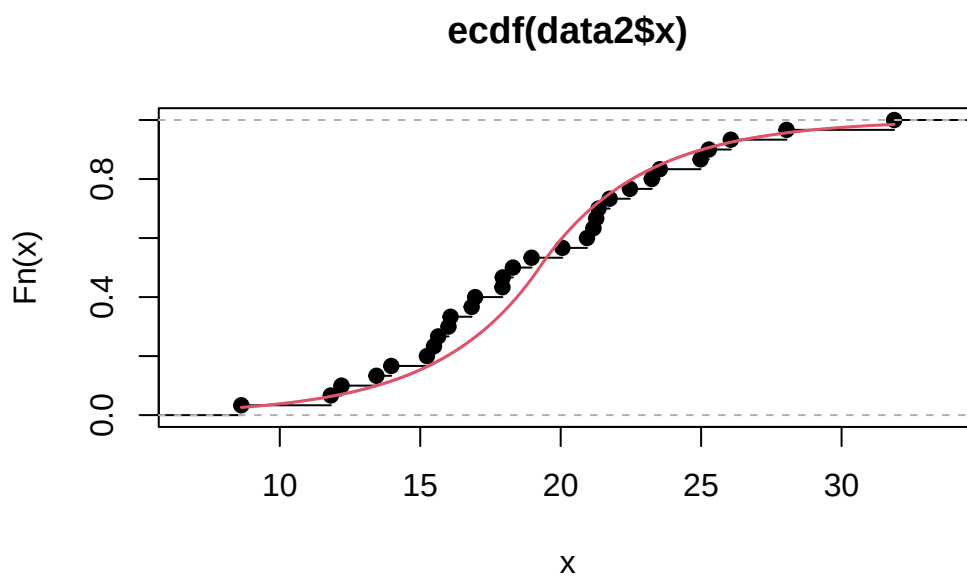
```
hist(data2$x, probability = TRUE)
```

```
x0 <- seq(min(data2$x), max(data2$x), 0.001)
```

```
lines(x0, dlaplace(x0, mu = mu.mme, sigma = b.mme), col = 2, lwd = 1.2)
```



```
plot(ecdf(data2$x))
lines(x0, plaplace(x0, mu = mu.mme, sigma = b.mme), col = 2, lwd=1.5)
```



Gamma distribution

- The decision variables for the normal distribution are α and λ .
- Objective function: $E(X) = \frac{\alpha}{\lambda}$ and $E(X^2) = \sigma^2 + E(X)^2 = \frac{\alpha(\alpha+1)}{\lambda^2}$
- Theoretical derivation / efficient algorithms:

$$\hat{\alpha}_{MME} = \frac{E(X)^2}{E(X^2) - E(X)^2}$$

$$\hat{\lambda}_{MME} = \frac{E(X)}{E(X^2) - E(X)^2}$$

Set1

```
n <- nrow(data1) # 樣本數量
EX <- mean(data1$x) # 一次動差
EX2 <- mean(data1$x^2) # 二次動差
alpha.mme <- EX^2/(EX2-(EX^2))
lambda.mme <- EX/(EX2-(EX^2))
cat("alpha.mme是:")
```

alpha.mme是:

```
print(alpha.mme)
```

```
[1] 0.0008640264
```

```
cat("lambda.mme是:")
```

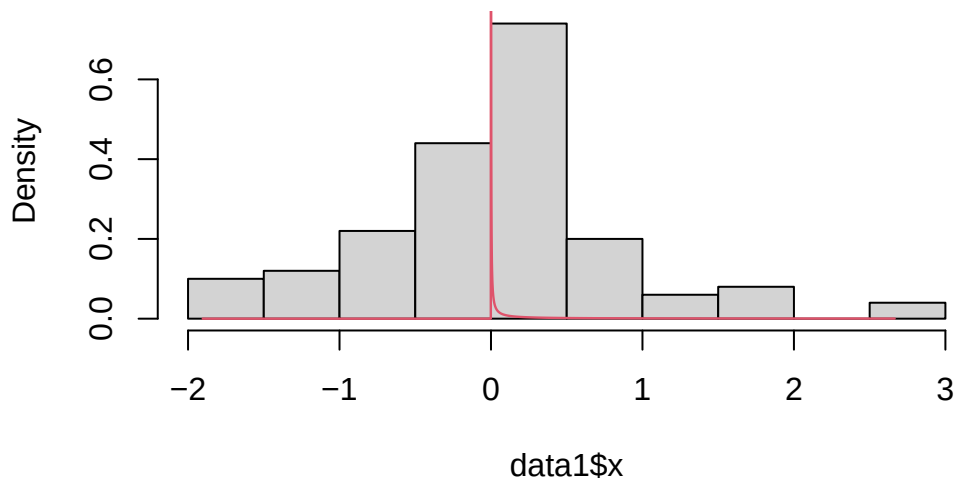
lambda.mme是:

```
print(lambda.mme)
```

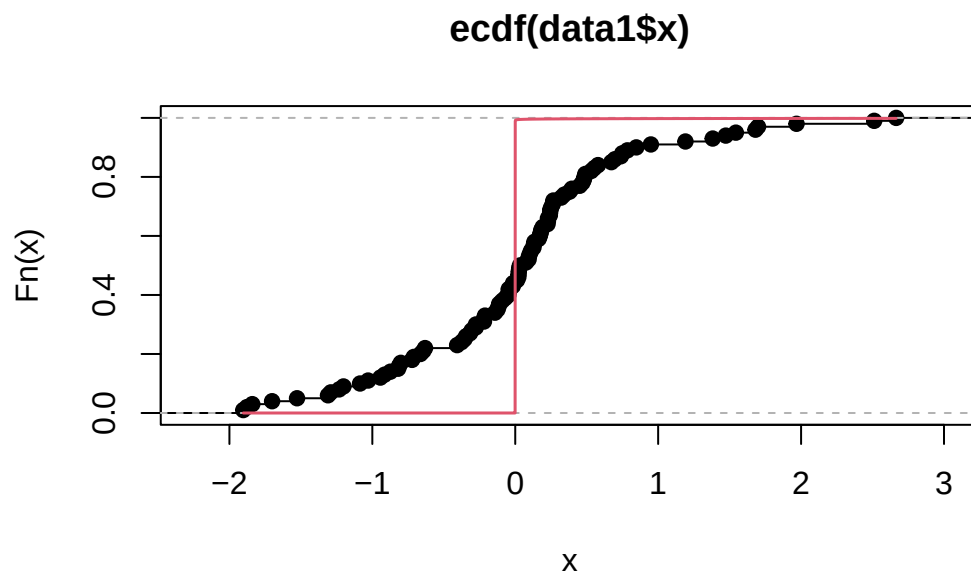
```
[1] 0.03496544
```

```
### Verification: By fitting plot
# install.packages("extraDistr")
library(extraDistr)
hist(data1$x, probability = TRUE)
x0 <- seq(min(data1$x), max(data1$x), 0.001)
lines(x0, dgamma(x0, shape = alpha.mme, rate = lambda.mme), col = 2, lwd = 1.2)
```

Histogram of data1\$x



```
plot(ecdf(data1$x))
lines(x0, pgamma(x0, shape = alpha.mme, rate = lambda.mme), col = 2, lwd = 1.5)
```



Set2

```
n <- nrow(data2) # 樣本數量
EX <- mean(data2$x) # 一次動差
EX2 <- mean(data2$x^2) # 二次動差
alpha.mme <- EX^2/(EX2-(EX^2))
lambda.mme <- EX/(EX2-(EX^2))
cat("alpha.mme是:")
```

alpha.mme是:

```
print(alpha.mme)
```

```
[1] 14.29557
```

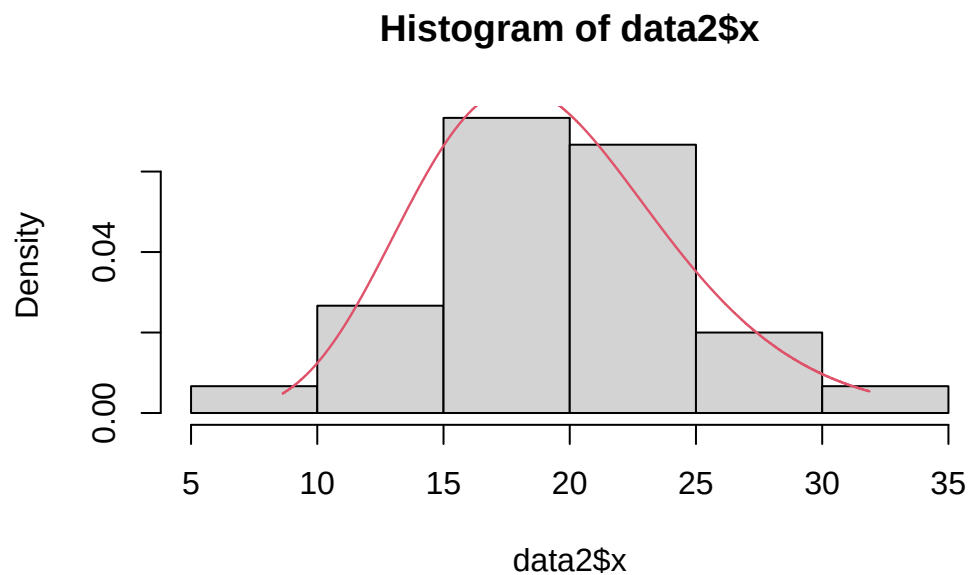
```
cat("lambda.mme是:")
```

lambda.mme是:

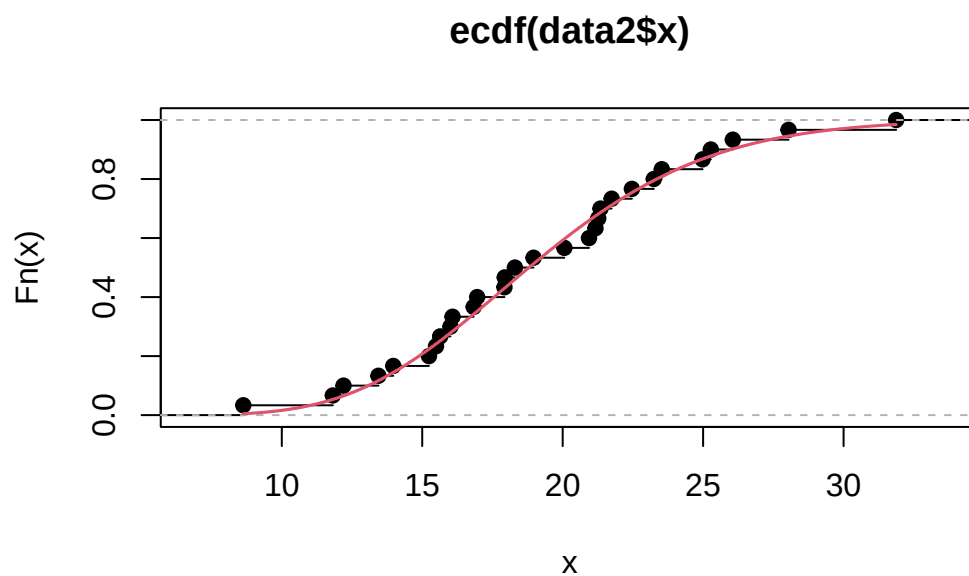
```
print(lambda.mme)
```

```
[1] 0.7427537
```

```
### Verification: By fitting plot
# install.packages("extraDistr")
library(extraDistr)
hist(data2$x, probability = TRUE)
x0 <- seq(min(data2$x), max(data2$x), 0.001)
lines(x0, dgamma(x0, shape = alpha.mme, rate = lambda.mme), col = 2, lwd = 1.2)
```



```
plot(ecdf(data2$x))
lines(x0, pgamma(x0, shape = alpha.mme, rate = lambda.mme), col = 2, lwd=1.5)
```



Conclusion

Table 1: MME to estimate the model parameters

Distribution	Dataset1	Dataset2
Normal	$(\hat{\mu}, \hat{\sigma}^2) = (0.025, 0.707)$	$(\hat{\mu}, \hat{\sigma}^2) = (19.247, 25.913)$
Exponential	$\hat{\lambda} = 40.468$	$\hat{\lambda} = 0.052$
Laplace	$(\hat{\mu}, \hat{b}^2) = (0.025, 0.594)$	$(\hat{\mu}, \hat{b}^2) = (19.247, 3.599)$
Gamma	$(\hat{\alpha}, \hat{\lambda}) = (0.00086, 0.03496)$	$(\hat{\alpha}, \hat{\lambda}) = (14.296, 0.743)$

(1.b) Use the maximum likelihood estimates (MLEs) to estimate the model parameters.

Normal

- The decision variables for the normal distribution are μ and σ^2 .
- Objective function: $\max_{(\mu, \sigma^2)} \prod_{i=1}^n f(x_i; \mu, \sigma^2)$
- Theoretical derivation / efficient algorithms:

$$\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i$$

and

$$\hat{\sigma}_{MLE}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

Set1

```
n <- length(data1$x)
mu.mle <- mean(data1$x)
sig2.mle <- sum((data1$x - mu.mle)^2)/n
```

```
### Numerical method ###
likelihood.normal <- function(par, data){
  mu <- par[1]
  sig2 <- par[2]
  joint <- dnorm(data, mean = mu, sd = sqrt(sig2))
  #return(prod(joint))
  #return(sum(log(joint)))
  return(-sum(log(joint)))
}
```

```
likelihood.normal(c(1, 2), data = data1$x) # -loglikelihood
```

```
[1] 167.999
```

```
likelihood.normal(c(mu.mle, sig2.mle), data = data1$x) # -loglikelihood
```

```
[1] 124.538
```

```
#optim(c(2, 1), likelihood.normal, data = data1$x, control=list(fnscale=-1))
mle.norm <- optim(c(2, 1), likelihood.normal, data = data1$x, hessian = TRUE)
```

```
### Verification: By hessian matrix
mle.norm
```

```
$par
[1] 0.02475207 0.70664295
```

```
$value
[1] 124.538
```

```
$counts
```

```
function gradient
  75    NA

$convergence
[1] 0

$message
NULL

$hessian
      [,1] [,2]
[1,] 141.514183280 -0.008250773
[2,] -0.008250773 100.155064164
```

```
det(mle.norm$hessian)
```

```
[1] 14173.36
```

Set2

```
### Theoretical method ###
n <- length(data2$x)
mu.mle <- mean(data2$x)
sig2.mle <- sum((data2$x - mu.mle)^2)/n
```

```
### Numerical method ###
likelihood.normal <- function(par, data){
  mu <- par[1]
  sig2 <- par[2]
  joint <- dnorm(data, mean = mu, sd = sqrt(sig2))
  #return(prod(joint))
  #return(sum(log(joint)))
  return(-sum(log(joint)))
}
```

```
likelihood.normal(c(1, 2), data = data2$x) # -loglikelihood
```

```
[1] 2729.378
```

```
likelihood.normal(c(mu.mle, sig2.mle), data = data2$x) # -loglikelihood
```

```
[1] 91.38912
```

```
#optim(c(2, 1), likelihood.normal, data = dat.x, control=list(fnscale=-1))
mle.norm <- optim(c(mu.mle, sig2.mle), likelihood.normal, data = data2$x, hessian = TRUE)
```

```
### Verification: By hessian matrix
mle.norm
```

```
$par
[1] 19.24671 25.91264
```

```
$value
[1] 91.38912
```

```
$counts
```

```
function gradient
  43    NA

$convergence
[1] 0

$message
NULL

$hessian
      [,1] [,2]
[1,] 1.157736 0.000000000
[2,] 0.000000 0.02233922

det(mle.norm$hessian)

[1] 0.02586292
```

Exponential

- The decision variable for the exponential distribution is λ .
- Objective function: $\prod_{i=1}^n \lambda e^{-\lambda x_i}$
- Theoretical derivation / efficient algorithms:

$$\hat{\lambda} = \frac{n}{\sum_{i=1}^n x_i}$$

Set1

```
### Theoretical method ###
n <- length(data1$x)
lambda.mle <- 1/mean(data1$x)
lambda.mle
```

```
[1] 40.46803
```

```
### Numerical method ###
likelihood.exp <- function(par, data){
  lambda <- par[1]
  joint <- dexp(data, rate = par)
  #return(prod(joint))
  #return(sum(log(joint)))
  return(-sum(log(joint)))
}
```

```
likelihood.exp(c(lambda.mle), data = data1$x) # -loglikelihood
```

```
[1] Inf
```

```
#optim(c(2, 1), likelihood.normal, data = dat.x, control=list(fnscale=-1))
mle.exp <- optim(c(40), likelihood.exp, data = data1$x)
```

Error in optim(c(40), likelihood.exp, data = data1\$x): function cannot be evaluated at initial parameters

從EDA中可以得知資料有負值，因此無法在此分布做MLE。

Set2

```
### Theoretical method ###
n <- length(data2$x)
lambda.mle <- 1/mean(data2$x)
lambda.mle
```

```
[1] 0.05195693
```

```
### Numerical method ###
likelihood.exp <- function(par, data){
  lambda <- par[1]
  joint <- dexp(data, rate = par)
  #return(prod(joint))
  #return(sum(log(joint)))
  return(-sum(log(joint)))
}
likelihood.exp(c(1), data = data2$x) # -loglikelihood
```

```
[1] 577.4013
```

```
likelihood.exp(c(lambda.mle), data = data2$x) # -loglikelihood
```

```
[1] 118.7202
```

```
#optim(c(2, 1), likelihood.normal, data = dat.x, control=list(fnscale=-1))
mle.exp <- optim(1, likelihood.exp, data = data2$x, method = "Brent", lower = 1e-6, upper = 10, hessian = TRUE)
```

```
### Verification: By hessian matrix
mle.exp
```

```
$par
[1] 0.05195693
```

```
$value
[1] 118.7202
```

```
$counts
function gradient
  NA    NA
```

```
$convergence
[1] 0
```

```
$message
NULL
```

```
$hessian
[1]
[1,] 11121.32
```

Laplace

- The decision variables for the normal distribution are μ and b .

- Objective function:

$$\prod_{i=1}^n \frac{1}{2b} e^{-\frac{|x_i - \mu|}{b}}$$

- Theoretical derivation / efficient algorithms:

$$\hat{\mu} = \text{median}(\{x_i\})$$

$$\hat{b} = \frac{1}{n} \sum_{i=1}^n |x_i - \hat{\mu}|$$

Set1

```
#### Theoretical method ####
n <- length(data1$x)
mu.mle <- median(data1$x) # Median for mu
b.mle <- mean(abs(data1$x - mu.mle)) # Mean absolute deviation for b
#### Numerical method ####
# Negative log-likelihood function for Laplace distribution
library(extraDistr)
likelihood.laplace <- function(par, data) {
  mu <- par[1]
  b <- par[2]
  if (b <= 0) return(Inf) # Ensure valid parameter range
  -sum(dlaplace(data, mu = mu, sigma = b, log = TRUE)) # Negative log-likelihood
}
init <- c(mu.mle, b.mle)
# Perform optimization
mle.laplace <- optim(
  par = init,
  fn = likelihood.laplace,
  data = data1$x,
  method = "L-BFGS-B",
  lower = c(-Inf, 1e-6), # Lower bounds
  upper = c(Inf, Inf), # Upper bounds
  hessian = TRUE # Compute Hessian matrix
)
# Extract results
mle.laplace
```

```
$par
[1] 0.05400467 0.59514085
```

```
$value
[1] 117.419
```

```
$counts
function gradient
  14    14
```

```
$convergence
[1] 0
```

```
$message  
[1] "CONVERGENCE: REL_REDUCTION_OF_F <= FACTR*EPSMCH"
```

```
$hessian  
      [,1] [,2]  
[1,]  0 0.000  
[2,]  0 282.337
```

Set2

```
### Theoretical method ###  
n <- length(data2$x)  
mu.mle <- median(data2$x) # Median for mu  
b.mle <- mean(abs(data2$x - mu.mle)) # Mean absolute deviation for b  
### Numerical method ###  
# Negative log-likelihood function for Laplace distribution  
library(extraDistr)  
likelihood.laplace <- function(par, data) {  
  mu <- par[1]  
  b <- par[2]  
  if (b <= 0) return(Inf) # Ensure valid parameter range  
  -sum(dlaplace(data, mu = mu, sigma = b, log = TRUE)) # Negative log-likelihood  
}  
init <- c(mu.mle, b.mle)  
# Perform optimization  
mle.laplace <- optim(  
  par = init,  
  fn = likelihood.laplace,  
  data = data2$x,  
  method = "L-BFGS-B",  
  lower = c(-Inf, 1e-6), # Lower bounds  
  upper = c(Inf, Inf), # Upper bounds  
  hessian = TRUE # Compute Hessian matrix  
)  
# Extract results  
mle.laplace
```

```
$par  
[1] 18.63171 4.14959
```

```
$value  
[1] 93.4847
```

```
$counts  
function gradient  
      3      3
```

```
$convergence  
[1] 0
```

```
$message  
[1] "CONVERGENCE: REL_REDUCTION_OF_F <= FACTR*EPSMCH"
```

```
$hessian
      [,1] [,2]
[1,] 0 0.000000
[2,] 0 1.742252
```

Gamma

- The decision variables for the normal distribution are α and λ .
- Objective function:

$$\prod_{i=1}^n \frac{\lambda^\alpha x_i^{\alpha-1} e^{-\lambda x_i}}{\Gamma(\alpha)}$$

- Gamma的參數估計(MLE)不好寫成close form

Set1

```
### Numerical method ###
# Negative log-likelihood function for Gamma distribution (rate parameterization)
library(extraDistr)
likelihood.gamma <- function(par, data) {
  alpha <- par[1]
  lambda <- par[2]

  if (alpha <= 0 || lambda <= 0) return(Inf) # Ensure valid parameter range
  # Negative log-likelihood (using dgamma from extraDistr package)
  -sum(dgamma(data, shape = alpha, rate = lambda, log = TRUE)) # Negative log-likelihood
}

# Initial guesses for parameters (alpha, lambda)
alpha.mle <- 2 # Example initial guess for alpha
lambda.mle <- 1 # Example initial guess for lambda
init <- c(alpha.mle, lambda.mle)
# Perform optimization
mle.gamma <- optim(
  par = init,
  fn = likelihood.gamma,
  data = data1$x,
  method = "L-BFGS-B",
  lower = c(0.1, 1e-6), # Lower bounds for alpha and lambda
  upper = c(Inf, Inf), # Upper bounds for alpha and lambda
  hessian = TRUE # Compute Hessian matrix for parameter uncertainty
)
```

Error in optim(par = init, fn = likelihood.gamma, data = data1\$x, method = "L-BFGS-B", : L-BFGS-B needs finite values for 'lower' and 'upper'

```
# Extract results
mle.gamma
```

Error: object 'mle.gamma' not found

因為data有負號 · 因此gamma不適合

Set2

```
### Numerical method ###  
# Negative log-likelihood function for Gamma distribution (rate parameterization)  
library(extraDistr)  
likelihood.gamma <- function(par, data) {  
  alpha <- par[1]  
  lambda <- par[2]  
  if (alpha <= 0 || lambda <= 0) return(Inf) # Ensure valid parameter range  
  # Negative log-likelihood (using dgamma from extraDistr package)  
  -sum(dgamma(data, shape = alpha, rate = lambda, log = TRUE)) # Negative log-likelihood  
}  
likelihood.gamma(c(2,1),data2$x)
```

```
[1] 489.7899
```

```
likelihood.gamma(c(13.693, 0.711),data2$x)
```

```
[1] 91.29147
```

```
# Initial guesses for parameters (alpha, lambda)  
alpha.mle <- 2 # Example initial guess for alpha  
lambda.mle <- 1 # Example initial guess for lambda  
init <- c(alpha.mle, lambda.mle)  
# Perform optimization  
mle.gamma <- optim(  
  par = init,  
  fn = likelihood.gamma,  
  data = data2$x,  
  method = "L-BFGS-B",  
  lower = c(0.1, 1e-6), # Lower bounds for alpha and lambda  
  upper = c(Inf, Inf), # Upper bounds for alpha and lambda  
  hessian = TRUE # Compute Hessian matrix for parameter uncertainty  
)
```

```
# Extract results  
mle.gamma
```

```
$par  
[1] 13.6931953 0.7114569
```

```
$value  
[1] 91.29139
```

```
$counts  
function gradient  
      22      22
```

```
$convergence  
[1] 0
```

```
$message  
[1] "CONVERGENCE: REL_REDUCTION_OF_F <= FACTR*EPSMCH"
```

```
$hessian
```

```

      [,1] [,2]
[1,] 2.272813 -42.16702
[2,] -42.167025 811.57865

```

```
det(mle.gamma$hessian)
```

```
[1] 66.50852
```

Conclusion

Table 2: MLE to estimate the model parameters

Distribution	Dataset1	Dataset2
Normal	$(\hat{\mu}, \hat{\sigma}^2) = (0.025, 0.707)$	$(\hat{\mu}, \hat{\sigma}^2) = (19.247, 25.913)$
Exponential	$\hat{\lambda} = X$	$\hat{\lambda} = 0.052$
Laplace	$(\hat{\mu}, \hat{b}^2) = (0.054, 0.595)$	$(\hat{\mu}, \hat{b}^2) = (18.631, 4.150)$
Gamma	$(\hat{\alpha}, \hat{\lambda}) = X$	$(\hat{\alpha}, \hat{\lambda}) = (13.693, 0.711)$

(1.c) Are the MMEs and MLEs the same for each distribution?

觀察以下2個table，發現在normal和exponential在不同estimation是一樣的。

Table 3: MME to estimate the model parameters

Distribution	Dataset1	Dataset2
Normal	$(\hat{\mu}, \hat{\sigma}^2) = (0.025, 0.707)$	$(\hat{\mu}, \hat{\sigma}^2) = (19.247, 25.913)$
Exponential	$\hat{\lambda} = 40.468$	$\hat{\lambda} = 0.052$
Laplace	$(\hat{\mu}, \hat{b}^2) = (0.025, 0.594)$	$(\hat{\mu}, \hat{b}^2) = (19.247, 3.599)$
Gamma	$(\hat{\alpha}, \hat{\lambda}) = (0.00086, 0.03496)$	$(\hat{\alpha}, \hat{\lambda}) = (14.296, 0.743)$

Table 4: MLE to estimate the model parameters

Distribution	Dataset1	Dataset2
Normal	$(\hat{\mu}, \hat{\sigma}^2) = (0.025, 0.707)$	$(\hat{\mu}, \hat{\sigma}^2) = (19.247, 25.913)$
Exponential	$\hat{\lambda} = X$	$\hat{\lambda} = 0.052$
Laplace	$(\hat{\mu}, \hat{b}^2) = (0.054, 0.595)$	$(\hat{\mu}, \hat{b}^2) = (18.631, 4.150)$
Gamma	$(\hat{\alpha}, \hat{\lambda}) = X$	$(\hat{\alpha}, \hat{\lambda}) = (13.693, 0.711)$

(1.d) and (1.e)

- 使用MME來找適合Dataset1和Dataset2的分布。
- 從(a)小題的圖可以看到Dataset1應該比較適合normal 或 laplace 而 Dataset2應該比較適合normal 或 gamma。

[plot for normal](#) [plot for laplace](#) [plot for gamma](#)

(1.f)

使用goodness-of-fit來檢定適合分布

dataset1

normal

```
### Goodness-of-fit ###
### Hypothesis Testing using theoretical cdf ###
# install.packages("goftest")
library("goftest")

n<-nrow(data1)# 樣本數量
EX<-mean(data1$x)# 一次動差
EX2<-mean(data1$x^2)# 二次動差
mu.mme<-EX
sigma2.mme<-EX2-(EX)^2

### K-S test ###
ks.N <- ks.test(data1$x, "pnorm", mean = mu.mme, sd = sqrt(sigma2.mme))
### AD test ###
ad.N <- ad.test(data1$x, "pnorm", mean = mu.mme, sd = sqrt(sigma2.mme))
### CVM test ###
cvm.N <- cvm.test(data1$x, "pnorm", mean = mu.mme, sd = sqrt(sigma2.mme))
# Summary #
dist.1 <- data.frame(normal = c(ks.N$p.value, ad.N$p.value, cvm.N$p.value))
```

laplace

```
library(extraDistr)
n <- nrow(data1) # 樣本數量
EX <- mean(data1$x) # 一次動差
EX2 <- mean(data1$x^2) # 二次動差
mu.mme <- EX
b.mme <- sqrt(0.5*(EX2-EX^2))

### K-S test ###
ks.L <- ks.test(data1$x, "plaplace", mu = mu.mme, sigma = b.mme)
### AD test ###
ad.L <- ad.test(data1$x, "plaplace", mu = mu.mme, sigma = b.mme)
### CVM test ###
cvm.L <- cvm.test(data1$x, "plaplace", mu = mu.mme, sigma = b.mme)
# Summary #
dist.1$laplace <- c(ks.L$p.value, ad.L$p.value, cvm.L$p.value)
kable(dist.1)
```

normal	laplace
0.2057896	0.9313387
0.1605055	0.8201209
0.1386345	0.7750835

dataset2

normal

```
n<-nrow(data2)# 樣本數量
EX<-mean(data2$x)# 一次動差
EX2<-mean(data2$x^2)# 二次動差
mu.mme<-EX
sigma2.mme<-EX2-(EX)^2

### K-S test ###
ks.N <- ks.test(data2$x, "pnorm", mean = mu.mme, sd = sqrt(sigma2.mme))
### AD test ###
ad.N <- ad.test(data2$x, "pnorm", mean = mu.mme, sd = sqrt(sigma2.mme))
### CVM test ###
cvm.N <- cvm.test(data2$x, "pnorm", mean = mu.mme, sd = sqrt(sigma2.mme))
# Summary #
dist.2 <- data.frame(normal = c(ks.N$p.value, ad.N$p.value, cvm.N$p.value))
```

gamma

```
n <- nrow(data2) # 樣本數量
EX <- mean(data2$x) # 一次動差
EX2 <- mean(data2$x^2) # 二次動差
alpha.mme <- EX^2/(EX2-(EX^2))
lambda.mme <- EX/(EX2-(EX^2))

### K-S test ###
ks.G <- ks.test(data2$x, "pgamma", shape = alpha.mme, rate = lambda.mme)
### AD test ###
ad.G <- ad.test(data2$x, "pgamma", shape = alpha.mme, rate = lambda.mme)
### CVM test ###
cvm.G <- cvm.test(data2$x, "pgamma", shape = alpha.mme, rate = lambda.mme)
# Summary #
dist.2$g <- c(ks.G$p.value, ad.G$p.value, cvm.G$p.value)
kable(dist.2)
```

normal	g
0.9926868	0.9381180
0.9983586	0.9991130
0.9897424	0.9955509

(1.g)

dataset1

用MLE之結果算log-likelihood 之值

```
# Load necessary libraries
library(MASS) # For fitdistr
library(stats4) # For AIC function if needed
```

```

n <- length(data1$x)
mu.mle <- mean(data1$x)
sig2.mle <- sum((data1$x - mu.mle)^2)/n

### Numerical method ###
likelihood.normal <- function(par, data){
  mu <- par[1]
  sig2 <- par[2]
  joint <- dnorm(data, mean = mu, sd = sqrt(sig2))
  #return(prod(joint))
  #return(sum(log(joint)))
  return(sum(log(joint)))
}

likelihood.n1 <- likelihood.normal(c(mu.mle, sig2.mle), data = data1$x) # loglikelihood
#####laplace
mu.mle <- median(data1$x) # Median for mu
b.mle <- mean(abs(data1$x - mu.mle)) # Mean absolute deviation for b
### Numerical method ###
# Negative log-likelihood function for Laplace distribution
library(extraDistr)
likelihood.laplace <- function(par, data) {
  mu <- par[1]
  b <- par[2]
  if (b <= 0) return(Inf) # Ensure valid parameter range
  sum(dlaplace(data, mu = mu, sigma = b, log = TRUE)) # Negative log-likelihood
}
likelihood.l1 <- likelihood.laplace(c(mu.mle, b.mle), data = data1$x) # loglikelihood

# Number of parameters
k_norm <- 2 # Normal: mean and sd
k_laplace <- 2 # Laplace: mean and scale

# Calculate AICs
AIC_norm <- -2 * likelihood.n1 + 2 * k_norm
AIC_laplace <- -2 * likelihood.l1 + 2 * k_laplace

# Compare results
results1 <- data.frame(
  Distribution = c("Normal", "Laplace"),
  LogLikelihood = c(likelihood.n1, likelihood.l1),
  AIC = c(AIC_norm, AIC_laplace)
)
kable(results1)

```

Distribution	LogLikelihood	AIC
Normal	-124.538	253.076
Laplace	-117.419	238.838

從結果來看，laplace表現的更好。

dataset2

用MLE之結果算log-likelihood 之值

```
# Load necessary libraries
library(MASS) # For fitdistr
library(stats4) # For AIC function if needed

n <- length(data2$x)
mu.mle <- mean(data2$x)
sig2.mle <- sum((data2$x - mu.mle)^2)/n

### Numerical method ###
likelihood.normal <- function(par, data){
  mu <- par[1]
  sig2 <- par[2]
  joint <- dnorm(data, mean = mu, sd = sqrt(sig2))
  #return(prod(joint))
  #return(sum(log(joint)))
  return(sum(log(joint)))
}

likelihood.n2 <- likelihood.normal(c(mu.mle, sig2.mle), data = data1$x) # loglikelihood
##### gamma
### Numerical method ###
# log-likelihood function for Gamma distribution (rate parameterization)
library(extraDistr)
likelihood.gamma <- function(par, data) {
  alpha <- par[1]
  lambda <- par[2]
  if (alpha <= 0 || lambda <= 0) return(Inf) # Ensure valid parameter range
  # Negative log-likelihood (using dgamma from extraDistr package)
  sum(dgamma(data, shape = alpha, rate = lambda, log = TRUE)) # log-likelihood
}
likelihood.g2 <- likelihood.gamma(c(13.6931953, 0.7114569), data = data2$x) # loglikelihood

k_norm <- 2
k_gamma <- 2
# Calculate AICs
AIC_norm <- -2 * likelihood.n2 + 2 * k_norm
AIC_gamma <- -2 * likelihood.g2 + 2 * k_gamma

# Compare results
results2 <- data.frame(
  Distribution = c("Normal", "Gamma"),
  LogLikelihood = c(likelihood.n2, likelihood.g2),
  AIC = c(AIC_norm, AIC_gamma)
)
kable(results2)
```

Distribution	LogLikelihood	AIC
Normal	-968.93812	1941.8762
Gamma	-91.29139	186.5828

從此結果來看，gamma分布應該比較適合dataset2。

iris data for question 2

Load datasets iris.csv

```
library(knitr)
iris <- read.csv("iris.csv"); head(iris)
```

```
Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1      5.1      3.5      1.4      0.2 setosa
2      4.9      3.0      1.4      0.2 setosa
3      4.7      3.2      1.3      0.2 setosa
4      4.6      3.1      1.5      0.2 setosa
5      5.0      3.6      1.4      0.2 setosa
6      5.4      3.9      1.7      0.4 setosa
```

EDA

5 Variables													iris	150 Observations		
Sepal.Length													Histogram			
n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95				
150	0	35	0.998	5.843	0.9462	4.600	4.800	5.100	5.800	6.400	6.900	7.255				
lowest : 4.3 4.4 4.5 4.6 4.7, highest: 7.3 7.4 7.6 7.7 7.9																
Sepal.Width													Histogram			
n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95				
150	0	23	0.992	3.057	0.4872	2.345	2.500	2.800	3.000	3.300	3.610	3.800				
lowest : 2 2.2 2.3 2.4 2.5, highest: 3.9 4 4.1 4.2 4.4																
Petal.Length													Histogram			
n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95				
150	0	43	0.998	3.758	1.979	1.30	1.40	1.60	4.35	5.10	5.80	6.10				
lowest : 1 1.1 1.2 1.3 1.4, highest: 6.3 6.4 6.6 6.7 6.9																
Petal.Width													Histogram			
n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95				
150	0	22	0.99	1.199	0.8676	0.2	0.2	0.3	1.3	1.8	2.2	2.3				
lowest : 0.1 0.2 0.3 0.4 0.5, highest: 2.1 2.2 2.3 2.4 2.5																
Species													Bar Chart			
n	missing	distinct														
150	0	3														
Value	setosa	versicolor	virginica													
Frequency	50	50	50													
Proportion	0.333	0.333	0.333													

(2.a)

```
n <- nrow(iris); n
```

```
[1] 150
```

```
petal.l <- iris[,3]  
mu.mle <- mean(petal.l) ; mu.mle
```

```
[1] 3.758
```

```
sig2.mle <- sum((petal.l - mu.mle)^2)/n ; sig2.mle
```

```
[1] 3.095503
```

```
### Numerical method ###  
likelihood.normal <- function(par, data){  
  mu <- par[1]  
  sig2 <- par[2]  
  joint <- dnorm(data, mean = mu, sd = sqrt(sig2))  
  #return(prod(joint))  
  #return(sum(log(joint)))  
  return(-sum(log(joint)))  
}  
likelihood.normal(c(mu.mle, sig2.mle), data = petal.l) # -loglikelihood
```

```
[1] 297.5871
```

```
likelihood.normal(c(3.757784, 3.095191), data = petal.l) # -loglikelihood
```

```
[1] 297.5871
```

```
#optim(c(2, 1), likelihood.normal, data = dat.x, control=list(fnscale=-1))  
mle.norm <- optim(c(2, 1), likelihood.normal, data = petal.l, hessian = TRUE)  
mle.norm
```

```
$par  
[1] 3.757784 3.095191
```

```
$value  
[1] 297.5871
```

```
$counts  
function gradient  
59 NA
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

```
$hessian  
      [,1] [,2]  
[1,] 48.462276908 0.003385495  
[2,] 0.003385495 7.830223026
```

By the MLE(Normal distribution), we can know that the parameters is:

$$\hat{\mu}_{MLE} = 3.758$$

$$\hat{\sigma}_{MLE}^2 = 3.095503$$

(2.b)

According to 2.a, we can know that $\hat{\mu}_{MLE} = 3.758$ by MLE method. This value is apparently greater than we assumed, 1.5.

(2.c)

By 2.a, we know that the sampling distribution of $\hat{\mu}$ is $\hat{\mu} \sim N(3.758, \frac{3.095503}{150})$.

I decide to use the bootstrapping method to complete this process. The code is shown below:

```
set.seed(24101222)
# 1. 定義參數
B <- 1000 # 自助抽樣次數
n <- length(petal.l) # 樣本大小 150
# 2. 初始化用於存放均值的向量
bootstrap_means <- numeric(B)
# 3. 開始自助抽樣
for (i in 1:B) {
  # 隨機抽樣 (有放回)
  resampled_data <- sample(petal.l, size = n, replace = TRUE)
  # 計算重抽樣後的均值
  bootstrap_means[i] <- mean(resampled_data)
}
# 4. 計算 95% 置信區間
ci_lower <- quantile(bootstrap_means, probs = 0.025) # 下界 ( 2.5% 分位數 )
ci_upper <- quantile(bootstrap_means, probs = 0.975) # 上界 ( 97.5% 分位數 )
# 5. 輸出結果
cat("95% Bootstrap Confidence Interval for Mean:\n")
```

95% Bootstrap Confidence Interval for Mean:

```
cat("Lower Bound:", ci_lower, "\n")
```

Lower Bound: 3.488

```
cat("Upper Bound:", ci_upper, "\n")
```

Upper Bound: 4.032717

According to the result shown above, the **95% confidence interval** of μ is $\mu \in [3.488, 4.032717]$

(2.d)

By the result shown in 2.c, the statement which said that **the mean of petal length is 1.5** is not reasonable.

(2.e)

$$t \text{ statistic} = \frac{\bar{x} - \mu}{s / \sqrt{n}}$$

```
# 1. 假設檢定參數
mu_0 <- 1.5 # 假設均值
alpha <- 0.05 # 顯著性水平

# 2. 計算 t 統計量
sample_mean <- mean(petal.l)
sample_sd <- sd(petal.l)
n <- length(petal.l)
t_stat <- (sample_mean - mu_0) / (sample_sd / sqrt(n)) # t 統計量

# 3. 計算自由度與 p 值
df <- n - 1 # 自由度
p_value <- 2 * pt(-abs(t_stat), df) # 雙尾檢定的 p 值

# 4. 檢查是否拒絕 H0
if (p_value < alpha) {
  cat("Reject H0: The mean is significantly different from 1.5.\n")
} else {
  cat("Fail to Reject H0: There is no significant evidence that the mean is different from 1.5.\n")
}
```

Reject H0: The mean is significantly different from 1.5.

```
# 5. 顯示結果
cat("t-statistic:", t_stat, "\n")
```

t-statistic: 15.66576

```
cat("p-value:", p_value, "\n")
```

p-value: 2.628305e-33

As the result shown in the code chunk, we reject the null hypothesis which said that the true mean is 1.5. This result have the same conclusion as the result in 2.d.